

Lab 17 - Volumes: emptyDir, hostPath and the Container Filesystem

Field	Value
Lab id	Lab 17
Course	TGS-2023039343 Kubernetes and Cloud Native Associate (KCNA) Training v6.0
Day / Topic	Day 4 Storage
Duration	45 minutes
Namespace	kcna-lab17
Maps to	LO4 Prepare a technical blueprint for a Kubernetes-based solution for security and storage A4 Prepare a technical blueprint for a solution in a given area K6 Technical blueprint design and construction process

1. Objective

By the end of this lab you will be able to:

1. Demonstrate that a container's writable layer is destroyed with the Pod, and state precisely which lifecycle event destroys it.
2. Author a multi-container Pod in which two containers exchange files through a single emptyDir volume, and explain why that is the only zero-config way to share a filesystem between containers.
3. Distinguish node-disk-backed emptyDir from emptyDir.medium: Memory (tmpfs), and show that sizeLimit is genuinely enforced.
4. Mount a hostPath volume with the correct type assertion and a readOnly mount, and articulate at least three concrete reasons hostPath is restricted or banned in production clusters.
5. Diagnose a FailedMount event caused by a hostPath type assertion, from kubectl describe pod alone.

2. Prerequisites

- * A single-node kind cluster running Kubernetes v1.30 or later, and a kubectl whose minor version is within one of the server.
- * kubectl configured with a context that can create namespaces.
- * Completion of Lab 04 (multi-container Pod patterns) and Lab 15 (ConfigMaps) - this lab assumes you can read a Pod spec fluently.
- * A terminal opened at this lab folder:

```
cd courseware/labs/lab-17-volumes-emptydir-hostpath
pwd
```

```
.../courseware/labs/lab-17-volumes-emptydir-hostpath
```

Confirm your cluster and client:

```
kubectl version
```

```
Client Version: v1.31.0
Kustomize Version: v5.4.2
Server Version: v1.31.0
```

Version strings will differ on your machine. Anything at v1.30 or above is fine for every step in this lab.

3. Scenario

Kallang Freight Pte Ltd is a Singapore logistics operator. Its platform team runs TrackLane, the shipment-tracking system that ingests bookings from three depots and publishes a rolling depot report to the operations floor.

Today the platform team has a specific complaint from operations: "the depot report page goes blank every time you deploy." The report is generated by a small builder process that writes an HTML file, and a web process that serves it - currently both baked into one container image, writing into the container's own filesystem. Nobody on the team can explain why the file disappears.

Your job for the next 45 minutes is to take that complaint apart. You will first reproduce the data loss, then rebuild the report pipeline as two containers sharing an emptyDir, then look at the two other places a Pod can put bytes on a node - a memory-backed tmpfs and the node's own filesystem via hostPath - and record for the platform blueprint exactly when each is appropriate.

You will carry this scenario through the whole of Day 4: Lab 18 gives the archive durable storage, Lab 19 gives the depot ledger a stable identity, Lab 20 looks at the control plane that runs it all, and Lab 21 packages it for delivery.

4. Step-by-step procedure

Step 1 - Create the lab namespace

Everything in this lab lives inside kcna-lab17. Nothing you do should touch another namespace.

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab17 created
```

Set it as the default for this shell so you cannot accidentally hit default:

```
kubectl config set-context --current --namespace=kcna-lab17
```

```
Context "kind-kcna" modified.
```

Confirm:

```
kubectl config view --minify -o jsonpath='{..namespace}{"\n"}'
```

```
kcna-lab17
```

Remember this. Every remaining command in this lab still passes -n kcna-lab17 explicitly so the lab is copy-pasteable even if you skipped this step.

Step 2 - Load the seed dataset into the cluster

data/shipments.csv holds 24 synthetic TrackLane bookings for 31 August 2026. Look at it first:

```
head -4 data/shipments.csv
```

```
shipment_id,booked_utc,origin_port,destination_port,mode,weight_kg,depot,status
KF-SG-100001,2026-08-31T02:14:00Z,SGSIN,MYPKG,road,1180,depot-a,in_transit
KF-SG-100002,2026-08-31T03:02:00Z,SGSIN,IDJKT,sea,24500,depot-b,at_port
KF-SG-100003,2026-08-31T03:47:00Z,SGSIN,THBKK,air,412,depot-a,cleared
```

Count the data rows (header excluded) - you will check the Pod's arithmetic against this number:

```
echo $(( $(wc -l < data/shipments.csv) - 1 ))
```

```
24
```

Project the file into the cluster as a ConfigMap:

```
kubectl -n kcna-lab17 create configmap tracklane-seed \
  --from-file=shipments.csv=data/shipments.csv
```

```
configmap/tracklane-seed created
```

```
kubectl -n kcna-lab17 get configmap tracklane-seed \
  -o jsonpath='{.data.shipments\.csv}' | head -2
```

```
shipment_id,booked_utc,origin_port,destination_port,mode,weight_kg,depot,status
KF-SG-100001,2026-08-31T02:14:00Z,SGSIN,MYPKG,road,1180,depot-a,in_transit
```

Step 3 - Reproduce the data loss (the container filesystem is ephemeral)

manifests/01-ephemeral-scratch.yaml declares no volume at all. Read it, then apply it:

```
kubectl apply -f manifests/01-ephemeral-scratch.yaml
```

```
pod/lane-scratch created
```

```
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-scratch --timeout=90s
```

```
pod/lane-scratch condition met
```

The container wrote a note into its own filesystem. Prove the file is there:

```
kubectl -n kcna-lab17 exec lane-scratch -- cat /tmp/tracklane/notes.txt
```

```
scratch-note written at 2026-09-05T04:11:23Z
```

Now capture the identity of the container instance, because that is what actually owns the bytes:

```
kubectl -n kcna-lab17 get pod lane-scratch \
-o jsonpath='{.status.containerStatuses[0].containerID}{"\n"}'
```

```
containerd://6f0a2b7c9e1d4a5f8c3b0e7d2a91c4f65b8e3d07a2c9f14b6e05d8371a2c9b4e
```

Delete the Pod and recreate it from the identical manifest:

```
kubectl -n kcna-lab17 delete pod lane-scratch --wait=true
```

```
pod "lane-scratch" deleted
```

```
kubectl apply -f manifests/01-ephemeral-scratch.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-scratch --timeout=90s
```

```
pod/lane-scratch created
pod/lane-scratch condition met
```

Read the note again:

```
kubectl -n kcna-lab17 exec lane-scratch -- cat /tmp/tracklane/notes.txt
```

```
scratch-note written at 2026-09-05T04:12:57Z
```

The timestamp changed. The old file did not survive; what you are reading is a brand-new file written by a brand-new container from the image. Confirm the container identity is different:

```
kubectl -n kcna-lab17 get pod lane-scratch \
-o jsonpath='{.status.containerStatuses[0].containerID}{"\n"}'
```

```
containerd://b31d7e05a9c8f24601e3b7d9c5a08f42e76b1c3d09a8f5e2c47b60d13f8a5c92
```

The mechanism. A container image is a stack of read-only layers. The runtime adds one thin writable layer per container, using an overlay filesystem. Every byte your process writes outside a mounted volume lands in that writable layer. When the container is removed, the runtime deletes the writable layer. A Pod restart caused by a crash keeps the Pod but replaces the container - so even a restartPolicy: Always crash-loop loses the data. Nothing about this is a bug; it is the reason volumes exist.

Clean up before the next step:

```
kubectl -n kcna-lab17 delete pod lane-scratch --wait=true
```

```
pod "lane-scratch" deleted
```

Step 4 - Read the emptyDir manifest before you apply it

```
grep -n -A 8 '^ volumes:' manifests/02-emptydir-shared.yaml
```

```
30: volumes:
31:   # The shared scratch space. No medium => node-backed (disk).
32:   - name: report
33:     emptyDir:
34:       sizeLimit: 64Mi
35:   # The seed dataset, projected read-only from a ConfigMap the learner
36:   # creates from data/shipments.csv.
37:   - name: seed
38:     configMap:
```

Three things to notice:

* emptyDir: {} (here with a sizeLimit) takes no identifying parameters. There is nothing to point at, because the volume has no existence before the Pod is scheduled - the kubelet creates an empty directory under /var/lib/kubelet/pods/<pod-uid>/volumes/kubernetes.io~empty-dir/report on whichever node wins the scheduling decision.

* Both containers list the same volume name in volumeMounts. That is the entire sharing mechanism.

* report-web mounts it with readOnly: true. The same volume can be read-write in one container and read-only in another; this is cheap least-privilege.

Step 5 - Share a filesystem between two containers with emptyDir

```
kubectl apply -f manifests/02-emptydir-shared.yaml
```

```
pod/lane-report created
```

```
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-report --timeout=120s
```

```
pod/lane-report condition met
```

```
kubectl -n kcna-lab17 get pod lane-report -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
lane-report	2/2	Running	0	18s	10.244.0.31	kcna-control-plane	<none>

2/2 - both containers are up. The builder logged what it wrote:

```
kubectl -n kcna-lab17 logs lane-report -c report-builder | head -1
```

```
report written to /shared/index.html (24 source rows)
```

24 source rows - matching the number you computed from data/shipments.csv in Step 2. The dataset really did travel ConfigMap → volume mount → builder → shared volume.

Now prove the *web* container can see a file the *builder* container wrote:

```
kubectl -n kcna-lab17 exec lane-report -c report-web -- cat /shared/index.html
```

```
<h1>TrackLane depot report</h1>
<p>built: 2026-09-05T04:15:02Z</p>
<p>pod: lane-report</p>
<p>rows: 24</p>
<p>in_transit: 7</p>
<p>held_customs: 2</p>
```

Serve it over the network to be certain the sharing is real and not a kubectl artefact:

```
kubectl -n kcna-lab17 port-forward pod/lane-report 8017:8080 >/tmp/pf-lab17.log 2>&1 &
sleep 3
curl -s http://127.0.0.1:8017/ | head -4
```

```
<h1>TrackLane depot report</h1>
<p>built: 2026-09-05T04:15:02Z</p>
<p>pod: lane-report</p>
<p>rows: 24</p>
```

Stop the port-forward:

```
kill %1 2>/dev/null; wait 2>/dev/null; echo "port-forward stopped"
```

```
port-forward stopped
```

Confirm the read-only mount is genuinely read-only from the web container:

```
kubectl -n kcna-lab17 exec lane-report -c report-web -- \
  sh -c 'touch /shared/tamper 2>&1 || echo "write refused (expected)''
```

```
touch: /shared/tamper: Read-only file system
write refused (expected)
```

Finally, look at how the node presents this volume. emptyDir with no medium is backed by whatever filesystem holds the kubelet root directory:

```
kubectl -n kcna-lab17 exec lane-report -c report-builder -- df -h /shared
```

Filesystem	Size	Used	Available	Use%	Mounted on
overlay	58.4G	21.7G	33.7G	39%	/shared

Reality check on sizeLimit for disk-backed emptyDir. For a disk-backed emptyDir the sizeLimit is **not** a filesystem quota - df still reports the whole node disk. It is an eviction threshold: the kubelet's ephemeral-storage monitor periodically measures the directory and evicts the Pod if it exceeds the limit. For a memory-backed emptyDir (Step 7) the limit **is** the tmpfs size and is enforced instantly. This asymmetry catches people out constantly.

Now delete the Pod and observe that the shared report is gone too:

```
kubectl -n kcna-lab17 delete pod lane-report --wait=true
kubectl apply -f manifests/02-emptydir-shared.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-report --timeout=120s
kubectl -n kcna-lab17 exec lane-report -c report-web -- cat /shared/index.html | grep built
```

```
pod "lane-report" deleted
pod/lane-report created
pod/lane-report condition met
<p>built: 2026-09-05T04:17:44Z</p>
```

A new build timestamp: emptyDir solved *container-to-container* sharing, but it did not solve durability. emptyDir shares the Pod's lifetime exactly. That is Lab 18's problem.

Step 6 - Locate the volume on the node (optional, kind only)

If you want to see the directory the kubelet created, and you have docker access to the kind node:

```
kubectl -n kcna-lab17 get pod lane-report -o jsonpath='{.metadata.uid}{"\n"}'
```

```
2f6c31ab-9e04-4b7d-8c15-7a0d3e59b6f1
```

```
docker exec kcna-control-plane \
  ls /var/lib/kubelet/pods/2f6c31ab-9e04-4b7d-8c15-7a0d3e59b6f1/volumes/kubernetes.io-empty-
  dir/report
```

```
heartbeat.txt
index.html
```

Substitute your own Pod UID and your own kind node container name (docker ps --format '{{.Names}}'). This step is illustrative only - skip it if you do not have Docker access. Never modify anything under /var/lib/kubelet.

Step 7 - Memory-backed emptyDir (tmpfs)

```
kubectl apply -f manifests/03-emptydir-memory.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/lane-cache --timeout=90s
```

```
pod/lane-cache created
pod/lane-cache condition met
```

```
kubectl -n kcna-lab17 logs lane-cache
```

```
--- mount type for /cache ---
tmpfs on /cache type tmpfs (rw,relatime,size=32768k,inode64)
--- df for /cache ---
Filesystem      Size      Used Available Use% Mounted on
tmpfs            32.0M          0      32.0M   0% /cache
```

Two facts are now on screen:

- * The filesystem type is tmpfs, not overlay. The pages live in the node's page cache rather than on the kubelet's disk. (Strictly, tmpfs pages *can* be swapped if the node has swap enabled; kubeadm and kind disable swap by default, but do not build a security argument on that assumption.)

- * The size is exactly 32 MiB - the sizeLimit you declared. The kubelet passed it to the tmpfs mount.

Write something small and read it back:

```
kubectl -n kcna-lab17 exec lane-cache -- \
  sh -c 'printf "lane-cache warm at %s\n" "$(date -u +%H:%M:%SZ)" > /cache/warm.txt; cat /
cache/warm.txt'
```

```
lane-cache warm at 04:19:31Z
```

When to use medium: Memory. Unix sockets between sidecars; short-lived material you would rather not leave on the kubelet's disk; hot caches where the disk round trip dominates. What it costs: tmpfs pages written by a container are charged to *that container's* memory limit, so you must budget the memory allowance for whatever you actually store - a container that fills a 1 GiB tmpfs needs a 1 GiB memory allowance. It is also still ephemeral: the volume dies with the Pod, and a node reboot wipes it.

Step 8 - Read the hostPath manifest before you apply it

```
grep -n -A 4 'hostPath:' manifests/04-hostpath-node-logs.yaml
```

```
36:     hostPath:
37:       path: /var/log
38:       type: Directory
39:   containers:
40:     - name: peek
```

The type field is an assertion the kubelet evaluates before mounting. The values you must know:

type	Kubelet behaviour
"" (empty, the default)	No check at all. Backwards-compatible and unsafe - a typo becomes a silently-created path.
DirectoryOrCreate	Create the directory (mode 0755, owned by the kubelet) if missing, then mount.
Directory	The path must already exist and be a directory, else the mount fails.
FileOrCreate	Create an empty file if missing, then mount.
File	The path must already exist and be a file.
Socket	The path must be an existing UNIX socket (this is how docker.sock gets mounted).
CharDevice / BlockDevice	The path must be an existing character / block device.

Step 9 - Mount the node filesystem read-only with hostPath

```
kubectl apply -f manifests/04-hostpath-node-logs.yaml
kubectl -n kcna-lab17 wait --for=condition=Ready pod/node-log-peek --timeout=90s
```

```
pod/node-log-peek created
pod/node-log-peek condition met
```

```
kubectl -n kcna-lab17 logs node-log-peek
```



```

--- top level of the node's /var/log (read-only) ---
containers
journal
kubelet.log
kube-apiserver.log
kube-controller-manager.log
kube-scheduler.log
pods
--- can we write to it? ---
read-only mount refused the write (expected)

```

You are looking at the node's filesystem from inside a Pod. containers/ and pods/ are the symlink trees the kubelet maintains - this is exactly what a Fluent Bit or Vector DaemonSet mounts to ship logs.

Now inspect one node-level path that only exists because you crossed the boundary:

```

kubectl -n kcna-lab17 exec node-log-peek -- \
sh -c 'ls /host-logs/containers | head -3'

```

```

etcd-kcna-control-plane_kube-system_etcd-9a4c1f7e0b3d2856ac9f4b17e05d3c82.log
kindnet-9tzqv_kube-system_kindnet-cni-4f0c8b2d19e7a536c04b81f3e9d27a05.log
kube-apiserver-kcna-control-plane_kube-system_kube-apiserver-3e07d5b1a982c46f0b73e15d8c204a9f.log

```

Why hostPath is restricted or banned in production. Four independent reasons, all of which have caused real incidents:

1. No isolation. A writable hostPath at / gives the container the node. Mounting /var/run/containerd/containerd.sock or /var/run/docker.sock gives it the ability to start a privileged container - a full node takeover in one API call.
2. No scheduling awareness. The scheduler does not know whether the path exists on a candidate node. The same Pod works on node-1 and hangs in ContainerCreating on node-2. You will see this in Failure Injection A.
3. No portability. The data is pinned to one node. Reschedule the Pod and it silently starts with different data, or none.
4. It bypasses your quota and your CSI story. Nothing counts hostPath bytes against a PVC, a StorageClass or a ResourceQuota.

>

The controls: the Pod Security Standards baseline and restricted profiles forbid hostPath volumes outright; policy engines such as Kyverno or a ValidatingAdmissionPolicy can allow a specific read-only allowlist for genuine node agents. If you think you need hostPath for application data, you almost certainly want a PersistentVolumeClaim - Lab 18.

5. Verification

Run the graded checks:

```

bash verification/checks.sh

```

```

== Lab 17 verification - namespace kcna-lab17 ==
[PASS] namespace kcna-lab17 exists
[PASS] configmap tracklane-seed carries 25 lines of seed data
[PASS] pod/lane-report is Running with 2/2 containers ready
[PASS] emptyDir volume 'report' is declared on pod/lane-report
[PASS] report-builder and report-web both mount the 'report' volume
[PASS] /shared/index.html reports 24 rows (matches data/shipments.csv)
[PASS] report-web sees the file written by report-builder (cross-container share)
[PASS] pod/lane-cache mounts a Memory-medium emptyDir with sizeLimit 32Mi
[PASS] /cache is a tmpfs sized 32.0M
[PASS] pod/node-log-peek mounts hostPath /var/log with type Directory
[PASS] the hostPath mount is readOnly
[PASS] no container in kcna-lab17 requests privileged: true
-----
12 passed, 0 failed

```

The full expected transcript is in verification/expected-output.md.

6. Failure injection

Failure A - hostPath type: Directory assertion fails

Apply the deliberately broken Pod:

```
kubectl apply -f manifests/05-hostpath-broken.yaml
```

```
pod/archive-writer-broken created
```

Wait ~20 seconds, then look:

```
kubectl -n kcna-lab17 get pod archive-writer-broken
```

NAME	READY	STATUS	RESTARTS	AGE
archive-writer-broken	0/1	ContainerCreating	0	22s

ContainerCreating that does not progress is almost always a volume problem. The events say so:

```
kubectl -n kcna-lab17 describe pod archive-writer-broken | tail -12
```

```

Events:
  Type     Reason      Age           From              Message
  ----     -
  Normal   Scheduled   24s           default-scheduler Successfully assigned kcna-lab17/
archive-writer-broken to kcna-control-plane
  Warning  FailedMount 8s (x7 over 24s) kubelet            MountVolume.SetUp failed for
volume "archive" : hostPath type check failed: /mnt/tracklane-archive is not a directory

```

Diagnosis, in order:

1. Scheduled succeeded - so this is *not* a scheduling, resource or affinity problem. The scheduler was happy; the kubelet is not.
2. FailedMount with (x7 over 24s) - the kubelet is retrying with backoff. It will retry forever; the Pod will never start on its own.

3. The message names the volume ("archive"), the failing check (hostPath type check), and the path (/mnt/tracklane-archive).
4. Cross-reference the manifest: spec.volumes[0].hostPath.type: Directory is an existence assertion, and /mnt/tracklane-archive does not exist on this node.

The three possible fixes, and which one is correct here:

Fix	Effect	Verdict
Change type to DirectoryOrCreate	Kubelet creates the directory; Pod starts	Works, but hides the real problem - you now have undeclared state on the node
<code>docker exec <node> mkdir -p /mnt/tracklane-archive</code>	Pod starts	Works on this node only; breaks the moment the Pod reschedules
Replace the hostPath volume with a PersistentVolumeClaim	Storage becomes a first-class, schedulable, portable API object	Correct. This is exactly what you build in Lab 18

Remove the broken Pod:

```
kubectl -n kcna-lab17 delete pod archive-writer-broken --ignore-not-found
```

```
pod "archive-writer-broken" deleted
```

Failure B - overfilling a memory-backed emptyDir

lane-cache has a 32 MiB tmpfs. Try to write 40 MiB into it:

```
kubectl -n kcna-lab17 exec lane-cache -- \
  sh -c 'dd if=/dev/zero of=/cache/blob bs=1M count=40'
```

```
dd: error writing '/cache/blob': No space left on device
32+0 records in
31+0 records out
33554432 bytes (32.0MB) copied, 0.041826 seconds, 765.2MB/s
command terminated with exit code 1
```

Diagnosis. ENOSPC at exactly 33554432 bytes = 32 MiB. The kubelet sized the tmpfs to sizeLimit, so the write failed at the filesystem layer *before* the Pod's memory cgroup was threatened. The Pod is still Running:

```
kubectl -n kcna-lab17 get pod lane-cache
```

NAME	READY	STATUS	RESTARTS	AGE
lane-cache	1/1	Running	0	4m12s

This is the good failure mode, and it is why you always set sizeLimit on a memory-backed emptyDir. Without it the tmpfs is sized by the node's default (traditionally half the node's RAM), so the same dd would have kept consuming real node memory until the container hit its own memory limit and was OOMKilled - or, if no memory limit were set, until the node itself came under memory pressure and the kubelet began evicting *other people's* Pods.

Reclaim the space:

```
kubectl -n kcna-lab17 exec lane-cache -- sh -c 'rm -f /cache/blob; df -h /cache'
```

Filesystem	Size	Used	Available	Use%	Mounted on
tmpfs	32.0M	0	32.0M	0%	/cache

7. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix	
Pod stuck ContainerCreating, event hostPath type check failed: <path> is not a directory	hostPath.type: Directory asserted on a path that does not exist on the assigned node	<code>`kubectl -n kcna-lab17 describe pod <pod> \</code>	<code>tail -15`</code>	Use a PVC instead; or DirectoryOrCreate if this really is node-local agent state
Pod stuck ContainerCreating, event configmap "tracklane-seed" not found	Step 2 skipped, or the ConfigMap was created in the wrong namespace	<code>kubectl -n kcna-lab17 get cm</code>	Re-run the create configmap command in Step 2 with -n kcna-lab17	
lane-report shows 1/2 and report-web is in CrashLoopBackOff	report-builder has not produced /shared/index.html yet, or crashed before writing it	<code>kubectl -n kcna-lab17 logs lane-report -c report-builder</code>	Read the builder's error; the web container waits for a non-empty file, so fix the builder first	
touch: /shared/...: Read-only file system from report-web	Working as designed - that container's volumeMount sets readOnly: true	<code>kubectl -n kcna-lab17 get pod lane-report -o j sonpath='{.spec.contai ners[1].volumeMounts}'</code>	Nothing to fix. Write from report-builder, which mounts it read-write	
Data written into /shared disappears after a deploy	emptyDir is bound to the Pod lifetime, not the workload	<code>kubectl -n kcna-lab17 get pod <pod> -o jsonp ath='{.spec.volumes}'</code>	Move the data to a PersistentVolumeClaim (Lab 18)	
df -h /shared shows the whole node disk despite sizeLimit: 64Mi	Expected: for disk-backed emptyDir, sizeLimit is an eviction threshold, not a quota	<code>kubectl -n kcna-lab17 get events --field-selector reason=Evicted</code>	If you need hard enforcement, use medium: Memory with sizeLimit, or a PVC with a real capacity	
Pod ... exceeds the limit "64Mi" Evicted event	The disk-backed emptyDir grew past sizeLimit and the kubelet evicted the Pod	<code>`kubectl -n kcna-lab17 describe pod <pod> \</code>	<code>grep -A3 Evicted`</code>	Raise sizeLimit, or stop writing bulk data into ephemeral storage

8. Cleanup

Delete only this lab's namespace. Every object you created in this lab (Pods and the ConfigMap) is namespaced, so one delete is complete and nothing outside kcna-lab17 is touched.

```
kubectl delete namespace kcna-lab17 --wait=true
```

```
namespace "kcna-lab17" deleted
```

Return your shell to the default namespace:

```
kubectl config set-context --current --namespace=default
```

```
Context "kind-kcna" modified.
```

Confirm nothing is left:

```
kubectl get namespace kcna-lab17
```

```
Error from server (NotFound): namespaces "kcna-lab17" not found
```

This lab created no cluster-scoped objects. No PersistentVolume, no StorageClass, no ClusterRole. The hostPath mount read the node's /var/log but wrote nothing to it - confirm with `kubectl -n kcna-lab17 logs node-log-peek` before you delete, which reports read-only mount refused the write.

9. What you learned

- * A container's writable layer belongs to the container, not the Pod and not the workload. Any container replacement - crash restart, image change, node reschedule - destroys it. You proved this with a changing timestamp and a changing containerID.
- * `emptyDir` is created by the kubelet when the Pod is bound to a node and deleted when the Pod is removed from that node. It is the correct and only zero-configuration answer to `"how do two containers in one Pod share files?"` - and it is the wrong answer to `"how do I keep this data?"`.
- * The same `emptyDir` can be mounted read-write in one container and read-only in another. Use that.
- * medium: Memory turns the volume into a tmpfs: RAM-backed, charged to the writing container's memory limit, and sized to `sizeLimit` by `SizeMemoryBackedVolumes` (beta from v1.22; GA in v1.32). Do not assume the node has swap disabled. For disk-backed `emptyDir` the same field is only an eviction threshold.
- * `hostPath` crosses the container boundary into the node. Its type field is an assertion evaluated by the kubelet at mount time, and getting it wrong produces a `FailedMount` event, not a scheduling failure - which is why describe pod beats get pod every time.
- * `hostPath` is forbidden by the baseline and restricted Pod Security Standards. Legitimate uses are node agents that mount a specific path read-only; application data belongs on a `PersistentVolumeClaim`.

Carry into Lab 18: you now have a concrete failure (`hostPath` type check failed) whose correct fix is `"make storage a first-class API object"`. Lab 18 builds exactly that.

10. Further reading

- * Kubernetes documentation - `*Volumes*`: <<https://kubernetes.io/docs/concepts/storage/volumes/>>
- * Kubernetes documentation - `*Volumes: emptyDir*`:
<<https://kubernetes.io/docs/concepts/storage/volumes/#emptydir>>
- * Kubernetes documentation - `*Volumes: hostPath*` (including the full type table and the security warning):
<<https://kubernetes.io/docs/concepts/storage/volumes/#hostpath>>
- * Kubernetes documentation - `*Ephemeral Volumes*`:
<<https://kubernetes.io/docs/concepts/storage/ephemeral-volumes/>>

* Kubernetes documentation - *Pod Security Standards* (hostPath is disallowed at baseline and above):
<<https://kubernetes.io/docs/concepts/security/pod-security-standards/>>

* Kubernetes documentation - *Node-pressure Eviction* (how sizeLimit is enforced for disk-backed volumes):
<<https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>>

* CNCF KCNA Curriculum - *Container Orchestration → Storage*: <<https://github.com/cncf/curriculum>>